

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/338690814>

Solving University Course Timetabling Problem Using Parallel Genetic Algorithm

Article in Nigerian Journal of Botany · October 2019

CITATIONS

8

READS

1,128

3 authors, including:



[Amin Rezaei Panah](#)

Dept. of Computer, Faculty of Computer

39 PUBLICATIONS 473 CITATIONS

[SEE PROFILE](#)

Some of the authors of this publication are also working on these related projects:



Masters Thesis: Providing an Innovative Method for Kicking Robotics [View project](#)

Solving University Course Timetabling Problem Using Parallel Genetic Algorithm

Amin Rezaeipanah^{1*}, Zahra Abshirini², Milad Boshkani Zade³

^{1*}Dept. of Computer, Faculty of Computer, University of Rahjuyan Danesh Borazjan, Bushehr, Iran

²Dept. of Applied Chemistry, Faculty of Science, Payame Noor University, Bushehr, Iran

³Dept. of Computer Engineering, Ahram Branch, Islamic Azad University, Ahram, Iran

*Corresponding Author: amin.rezaeipanah@gmail.com, Tel.: +98-9351350563

Available online at: www.isroset.org

Received: 08/Oct/2019, Accepted: 16/Oct/2019, Online: 31/Oct/2019

Abstract—Scheduling is one of the problems which so many researches have been conducted on it over the years. The university course timetabling problem (UCTP) which is an NP-Hard problem is a type of scheduling problem. The allocation of whole of events in timeslots and rooms performs by the university course timetabling process considering the list of hard and soft constraints presented in one semester, so that no conflict is created in such allocations. **In general, it means assigning predefined courses to certain rooms and timeslots under specific constraints.** In this paper we establish a hybrid algorithm based on Parallel Genetic Algorithm and Local Search to solve course timetabling problem (PGALS). This combines a direct representation of the timetable with heuristic crossover operators to ensure that the most fundamental constraints are never violated. We see how the algorithm is guaranteed to always produce a feasible solution by hard coding constraints which must not be broken. The proposed algorithm has been applied and evaluated against the latest methodologies in the literature with respect to standard benchmark problems. We demonstrate that the proposed algorithm produces some of the best-known results when tested on BenPaechter competition datasets.

Keywords— Parallel Genetic Algorithm; Local Search; Timetabling Problem; Hybrid Algorithm; BenPaechter Dataset.

I. INTRODUCTION

The process of planning and timetabling courses in universities is a multi-stage process requiring appropriate coordination and communication among several planning groups. These groups include university professors, faculty group heads, educational department manager and planners [1]. The University Course Timetabling Problem (UCTP) is one of the controversial issues presented in various applications. The UCTP goal is to create a timetable for a classroom program, so that there is no interference in the classroom and student timing program. The final program should also be optimal, which means that the best classes and best hours of courses are available. The best choices depend on the parameters specified for the problem [2]. This problem lies in the class of NP-Hard problems in terms of complexity and to find an optimal solution evolutionary algorithms such as the genetic algorithm are very effective. In a timing issue, there are many constraints. Some of these constraints must necessarily be respected, which are called hard constraints. Observing some of the other constraints will also improve the program. These constraints are soft

constraints. However, failure to comply with these constraints does not make the answer to the problem unacceptable, but it is better to follow them. The compliance with these constraints leads to approaching the optimal response. Most systems designed in this field use classical algorithms for classroom programming. Since classical algorithms provide an acceptable answer rather than an optimal answer, new approaches to timing issues are evolutionary and metaheuristic algorithms.

Over the past 40 years, researchers have been using different time-tabling methods using constraint-based methods, population-based methods (eg, genetic algorithms [3], [4], ant colony optimization [5], and memetic algorithms [6]), the metaheuristic methods (eg, tabu search [7], simulated annealing [8], and great deluge [9]), variable neighbourhood search (VNS), and hybrid and hyper-hybrid approaches, etc., have been proposed. In this paper a new hybrid approach based on parallel genetic algorithm and local search is utilized in solving university course timetabling problem. Because of simultaneous studying of several answers and random survey of case space which is perfect for the

scheduling problem, The preferred method in this study is the genetic algorithm.

The remainder of the article is as follows: In section II, it is discussed about the university course timetabling problem, whereas Section III briefly reviews preliminaries the related works. Section IV describes the proposed approach. Section V describes the results and discussions. Finally, in Section VI, the conclusion suggests possible future work is presented.

II. UNIVERSITY COURSE TIMETABLING PROBLEM

BenPaechter's dataset, one of the most famous collections of university course timetabling, has been used to conduct experiments in this research. This dataset was created in 2005 by Ben Paechter and colleagues and was available at <http://code.ulb.ac.be>. This dataset consists of 12 samples in three small, medium and large sizes. Characteristics of each category are given in Table 1 [10]. This problem includes scheduling 100-400 courses into a timetable with 45 timeslots similar to 5 days of 9 hours as well as satisfying room features and room capacity constraints.

The dataset in Table 1 contains four columns, in which the first column is a list of department events and resource classifications, and the other three columns represent the number of events and resources in small, medium and large sizes. The row number of this dataset is related to each event and the resources available in the department are in ascending order from small to medium to large.

Table 1. Details of BenPaechter dataset

Features	Categories		
	Small	Medium	Large
Number of events	100	400	400
Number of rooms	5	10	10
Number of room features	5	5	10
Approx. features per room	3	5	5
Percent feature use	70	80	90
Number of student	80	200	400
Max. events per student	20	20	20
Max. student per events	20	50	100

Constraints on the UCTP issue are divided into hard and soft constraints. Hard constraints, in contrast to soft constraints, should not be violated in any way. There is no obligation to satisfy soft constraints, but the quality of the timetable increases with increasing observing soft limits. Below is a list of hard and soft constraints taken from the literature of Al-Betar et al. [11].

Hard Constraints:

- H_1 : No student can be assigned more than one course at the same time.

- H_2 : The room should satisfy the features required by the course.
- H_3 : The number of students attending the course should be less than or equal to the capacity of the room.
- H_4 : No more than one course is allowed for each time slot in each room.

Soft Constraints:

- S_1 : A student should not have a single course on a day.
- S_2 : A student should not have more than two consecutive courses.
- S_3 : A student should not have a course scheduled in the last time slot of the day.

III. RELATED WORK

In recent years, researchers turn their focus on meta-heuristic, hyper-heuristic and hybridization methods. Several examples on these methods including hill climbing, simulated annealing, great deluge, genetic algorithm, particle swarm optimization, and harmony search. In this paper we are using a multi-stage approach. Although different to hybridized approaches, there are good reasons for using multiple algorithms within an overarching approach. Hybridization has led to good quality results in previous research [12]. More recent work has validated these earlier findings. For example, hybridized mixed integer linear programming, a greedy heuristic, two local search strategies and three meta-heuristics for a vehicle routing problem, reporting positive results [13]. The GA was used as an exploration operator, SA was used to intensify the search. The algorithm was evaluated on 35 cell formulation benchmark instances, producing 24 best results, and two new best results.

Many variants of the high school timetabling problem and their attempts to solve them can be found in respective references [14]. Their fundamental differences are attributed to the uniqueness of each country's educational system. For example, Moura and Scaraficci solve the high school timetabling problem for three brazilian high schools using a basic Greedy Randomized Adaptive Search Procedure (GRASP) heuristic followed by a path-relinking [15]. Another hybrid technique which combined two population-based algorithms for real-world course timetabling (i.e. bee colony with hill climbing optimizer) by Bolaji et al. [16]. Daskalaki et al. compared the performance of Genetic Algorithms (GAs) as opposed to Genetic Programming (GP) in solving a set of hard high school timetabling problems [17]. Da Fonseca et al. presented a local search approach to the high school timetabling problem [18]. Ahmed et al. evaluated the performance of a range of selection hyper-heuristics combining different reusable components for high school timetabling [19]. Al-Yakoob and Sherali investigated two decomposition approaches to solve a high school timetabling problem in a case study related to kuwait's public educational

system [20]. Lewis and Thompson achieved 100% feasibility on all instances of ITC2007 by using constructive heuristics and followed by their PARTIALCOL algorithm which uses a tabu mechanism for the remaining unassigned events [21].

Hyper-heuristics have emerged as such general purpose, high-level search methodologies which are motivated by the goal of selecting or generating heuristics automatically to solve a wide range of difficult optimisation problems [22]. This work focuses on the selection type of hyper-heuristics.

Most studies conducted in this area are single-objective in which minimizing violation of soft constraints is considered as objective function, and few studies have been focused on optimizing multi-objective models and multi objective optimization approaches. Datta, Deb [23] presented a bi-objective model with the aim of minimizing average number of free time slots between two classes of students and number of consecutive classes for lecturers. Then, they developed a NSGA-II algorithm for solving the model. In another study, Abdullah, Turabieh [24], presented a two-objective model to minimize the number of time slots during which a student waits between two classes as well as the deviations of soft limitations.

Timetabling problems as well as scheduling problems, define a class of hard-to-solve constrained optimization problems of

combinatorial nature. In order to solve the model, they developed a NSGA-II algorithm. Thepphakorn, Pongcharoen [25] developed a new multi-objective model by the aim of minimizing the operational costs and the number of inadequate chairs in a course time tabling problem in a university in Thailand. Then, to solve it, they used a multi objective cuckoo search algorithm.

IV. METHODOLOGY

In this paper a new approach is provided for the university course timetabling problem. In our proposed PGALS, timetables are created from hybridization of genetic algorithm and local search. The details of each operator and section will be discussed in the following sections. Because of simultaneous studying of several answers and random survey of case space which is perfect for the scheduling problem, The preferred method in this study is the genetic algorithm. After applying the standard genetic algorithm On different samples and answers, It was concluded that this method needs to improve due to the random search on the entire case. As a result, a new approach was created to solve the problem by making some reforms in the algorithm, such as restorative procedures on children, Intelligent operator and the use of parallel structure. The pseudo code for the PGALS algorithm as depicted in Figure 1. The following steps explain the proposed algorithm to solve this problem.

PGALS Algorithm: *Parallel Genetic Algorithm And Local Search*

Input: A instance of BenPaechter dataset.

Output: Timeslot and room number on each event.

ParFor $i = 1$ to MP *DO* // MP is number of parallel

Parallel Initialization: The number of population produced random chromosome(ch), according to Table 2.

Parallel Evaluation: Calculate fitness function for each solution, $f(ch)$, base on the eq.(1).

For $j = 1$ to Max Generation *Do*

Select two parents with roulette wheel selection operator, (p_1, p_2) .

Apply crossover operator and production three children base on the crossover rate, (c_1, c_2) .

Apply improvement operator on the two children, (ic_1, ic_2) .

Calculate fitness function for two children, $f(ic_1)$ and $f(ic_2)$, and computing S_1, S_2, S_3 and P_{dis} .

If $P_{dis} = 0$ *Then*

Apply three mutation operator base on the mutation rate (mc_1, mc_2) .

Else

Apply local and global mutation operator base on the mutation rate only not specified events.

EndIf

Calculate fitness function for two children, $f(mc_1)$ and $f(mc_2)$.

Best Chromosome = Chromosome selected with the minimum fitness in between all MP population.

EndFor

The best chromosomes exchange between MP populations.

Add children to the population of the next generation if beter, for each mp population.

EndParFor

Applay local search with max iteration on the Best Chromosome.

Figure 1. Pseudo-code of PGALS algorithm

A. Genetic Algorithm

Chromosomes length in the university courses scheduling problem is related to the number of events. On the other hand, each event includes two genes characterized by timeslot (time event) and room numbers. Room numbers can be determined according to the number of input sample rooms. According to the BenPaechter standard, class timeslot have values between 1 to 45 (9 hours per day) in 5 days. A sample of proposed chromosomes is shown in Table 2.

In this study we will select the chromosomes in the form of random from search space. The evaluation criteria in the issue of scheduling the university courses are violation of issue constraints.

Table 2. Structure of proposed chromosome (for example)

#Events	1	2	3	4	5
#Timeslot	-1	34	-1	44	3
#Room	-1	2	-1	6	7

Fitness function measures the value of their violations by studying the limitations of hard and soft issue and according to which determines the suitability of chromosomes. A weight will be determined for each of the limitations which expresses the level of importance for each one. In this study, a criteria such as distance to feasibility is used in order to calculate the quality loss on schedule. Therefore, the required number of students are detected to participate in any infeasible event, and the total number of the students will give us the distance to feasibility schedule. For example, if a chromosome has three unallocated events and the number of students in each of these events are respectively 12, 8 and 5. So, the distance to feasibility is (5+8+12) 25. Eventually, the fitness function is calculated by following equation.

$$F(X) = \sum_{i=1..n} w_i^s \times P_i^{Soft} + \sum_{j=1..n} w_j^h \times P_j^{Hard} + w^d \sum_{k=ENS} P_k^{Dis} \quad (1)$$

In this equation, w the weight (penalty) can be any constraints (soft and hard). Weight constraints will be determined according to its importance in the timetable. P^{Soft} is the number of defects in soft constraints, P^{Hard} is the number of defects in hard constraints, P^{Dis} is the distance to the feasibility of solution, ENS (Events Not Specified) is the events that are not allocated to the timetable. Note that the proposed method prevents the creation of events with hard constraints and it considers them as the events which are not allocated. So in the above equation, the number of hard constraints is always zero. In recent years, how to deal with chromosomes that have this type of constraints is one of the main challenges. The aim of this procedure is to repair the chromosomes which have such constraints.

In this study, we select two parents according to the roulette wheel method in order to help convergence of the algorithm and the use of chromosomes of other populations. The first parent is selected from the population of the current genetic algorithm. Furthermore, the second parent is probably selected from the population of other sectors for 50% and is probably selected from the best produced chromosome for 50%.

The pairs which were considered as a parent in the selection part, they exchange their genes. In this part and create new members. If the crossover action does not occur on a chromosome pair, the children will be produced by repeating the parents. In this study, uniform crossover operator is designed that each produce one child. In the uniform crossover operator, at first, the child chromosome is generated by '0' and '-1' genes randomly. Then, for every genes with the content of '0' on children chromosome, the equivalent genes are copied from the first parent. In the next step instead of '-1' gene, the genes will be copied from the second parent that do not create hard constraints for child chromosome. Finally, a number of genes may not be assigned any values, these genes have '-1' content and are considered as infeasible events. An example of this operator can see in Figure 2.

Mutation is a random process in which the content of a gene is replaced by another gene in order to produce a genetic structure. This research provides two operators called Local Mutation, Global Mutation. At local mutation operator, one event is randomly selected within the chromosome for infeasible individual events, and the events are swap if fitness improvement is observed.

Parent 1						Parent 2					
7	14	5	2	3	4	5	6	14	13	8	5
3	1	2	3	1	2	4	1	1	3	4	5
Initial genes child						Child					
-1	0	-1	0	0	-1	5	14	-1	2	3	4
-1	0	-1	0	0	-1	4	1	-1	3	1	2

Figure 2. An example of uniform crossover operator

Swap genes are probably 50% of the timeslot and room number, and probably 25% of only timeslot and 25% probably only room number. Infeasible events are those of events that create constraint. Furthermore, in global mutation operator, first a list of all feasible events is determined to join the chromosome. The events can be obtained by sharing the current chromosomes events and all feasible states for the event. In the next stage, an event is randomly selected from the list of possible events for each current and infeasible chromosome event, in a way that does not create hard constraints. The new events are replaced if fitness improvement of the chromosome is observed.

Some of the timetable events in the first stages are not programmable. These events were indicated by '-1' in the genes of a chromosome. The purpose of improvement operator in the proposed algorithm is finding timeslot and room number in the timetable for this type of events so that the proposed solution is infeasible. The operator in fact is designed to improve the timetable and reduce the distance to the feasibility study on child chromosome. The operator randomly allocates Timeslot and room number (among all events state) to the events that are contradictory. One of the interesting phenomena in genetic algorithms is that it may be produced chromosomes in median generations which are very suitable and well-being at fitness function quality. The chromosomes may be destroyed in the result of the combination and mutation operators and will not be produced anymore. One way is to identify such cases and also use them in subsequent generations. This technique is called elitism because it has a great influence to answer the question practically. In this study, the best chromosome in each generation is selected among all populations and directly goes to the next generation.

B. Local Search

Local search is introduced to find a solution in order to maximize a criterion among a number of ongoing solutions. These algorithms move from one solution to another one in a space of ongoing solutions (search space) by using limited changes, until a desirable solution is found or a period of time passes. There may be cases in the genes of a chromosome which are less repeated or have not been used at all. For this reason, it requires the operator to enhance the probability of unobserved selection characteristics. The local search, swap randomly some of the genes on unused child chromosome by child chromosome gene. If the quality (fitness function) of timetabling increases, the changes can be done. The termination of the local search algorithm could be based on a timeslot when the best solution has not been optimized by the algorithm in the definite steps.

C. Structure of the Parallel Genetic Algorithm

One of the important features of the genetic algorithm is to be run in a parallel way and also to search for spaces that are very complex or large. In general, it can be argued that the issues where the search space is very large and complex, in local optimum the possibility of entrapment for parallel genetic algorithms is so low. This research tries to create different populations instead of using an initial population and processing parallel genetic algorithms. Chromosome structure is identical in all populations. But the population could have the selection operator, crossover and mutation of its own. Another advantage of this method is the possibility of parallel processes in each population, independent of the others. So, in this study producing the initial population and fitness function calculation is done in a parallel way. After a few generations, we will analyze the chromosomes exchange by

elitist algorithm. In the exchange of chromosomes, the worst chromosomes in current population are replaced by the best chromosomes in other populations (based on the fitness function).

Care must be taken in the exchange of chromosomes, such that, the number of chromosomes of each population should be greater than the number of new imported chromosomes. By observing this point, we can prevent the incompatible methods to impact on final result negatively. The main problem of convergence genetic algorithm is the local maximum (minimum) which is resulted from similarity of chromosomes (unable to create new and different generations of the same parents). Mutation step due to a random change has little reliability. But the existence of different chromosomes with high suitability (for creating a new and diverse generation) reduces the possibility of convergence to the local maximum (minimum). In this issue, because of variety of constraints, there is no possibility of designing the high-performance genetic algorithm which always solves the problem. Multi-population genetic algorithm tries to resolve (make it ineffective) this problem.

V. RESULTS AND DISCUSSION

In order to evaluate the effectiveness of the proposed approach in timetabling of courses, a program is designed and implemented in MATLAB R2016a language, and the experiments were conducted on a PC with an Intel Core i3, 2.2 GHz, 8GB RAM, Windows 10 ultimate. In the proposed algorithm, the weights of all the Hard constraints have been considered 0.9. This high weight will prevent the selection of chromosomes with hard constraints. Noting that the present issue is a research and soft constraints does not have any implementation priority, so the weight of the three soft constraints is considered identical and 0.1. Proposed input parameters of the suggested algorithm (PGALS) as follows: Population size is 50; Crossover rate is 0.8; Mutation rate is 0.15; Max generation is 75; Number of parallel is 5, Maximum local search iteration is 1000.

For showing the amount of proposed method, the suitability of chromosomes (constraints violated) have been investigated compared to the number of successful repetitions. Figure 3 to 5 show the convergence of the proposed algorithm on small, medium and large instances respectively. As it is obvious, in the first step, the proposed algorithm quickly reduces the accuracy of the DF (distance to feasibility). This is due to the structure of mutation operator and its strategy that is responsible for dealing with such chromosomes. In the small instances examples such as small in which the amount of DF is at first is zero, the structure of algorithm leads to pressure on the soft constraints and causing a rapid decline in their value at the first steps.

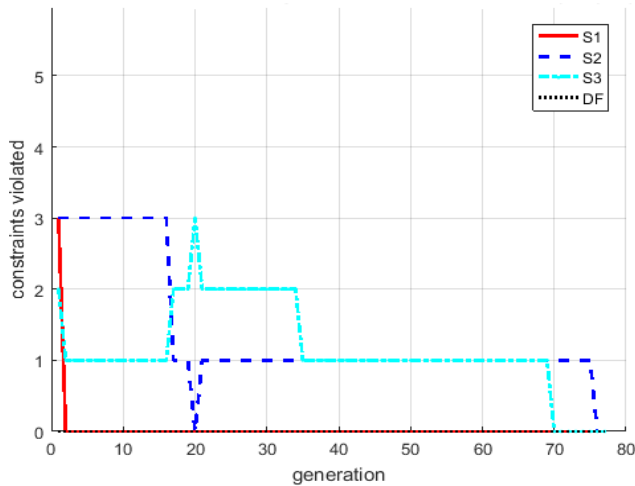


Figure 3. Convergence of the proposed algorithm on small1 instance

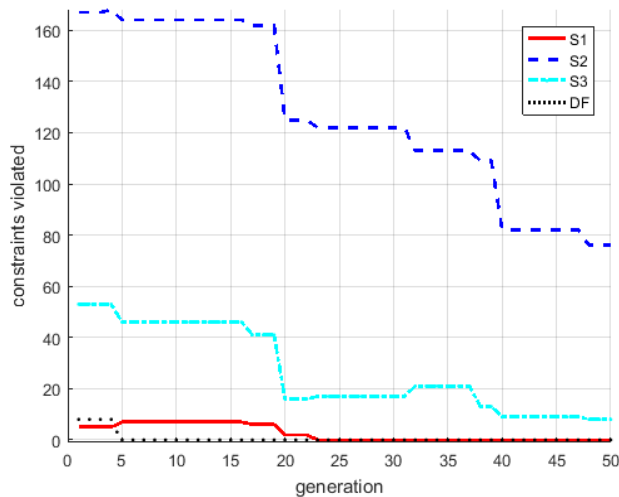


Figure 4. Convergence of the proposed algorithm on medium1 instance

The DF value of convergence charts in small instances is zero. In these instances, due to the small size of the search space and the proposed greedy algorithm, the hard limitations are not created genes and the chromosomes are not '-1' gene at the beginning.

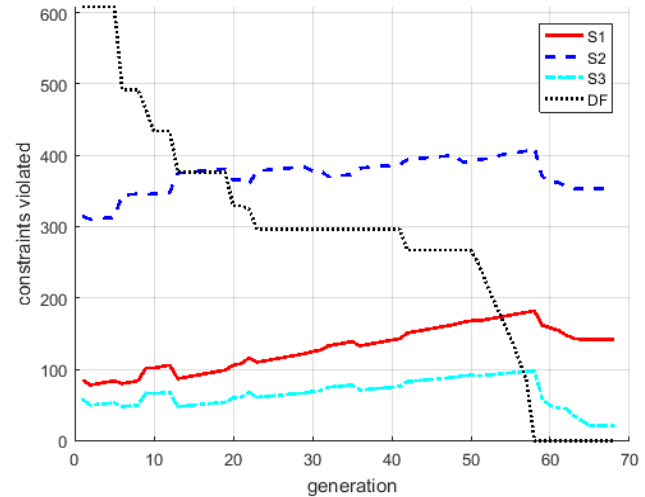


Figure 5. Convergence of the proposed algorithm on large1 instance

Figure 6.

Therefore, operators focus on minimizing soft constraints. But the DF values are not zero in the case of the medium and large because there are large dimensions of problem. In this case, according to the provided operators and algorithms Strategy against DF parameter, the proposed algorithm focuses more on reliability of solution. As clearly shown in Figure 4 and 5, DF value compared to other three parameters declines more quickly. When the parameter is zero, the algorithm focuses on soft constraints of the problem. S_2 constraints value is higher than any other constraints. Due to the high number of students and lessons for each student, it seems reasonable to say that the constraint compared to other constraints is in a high value.

Table 3 compares the performance of the proposed algorithm to other similar ones. In this table, the terms B and M, respectively, meaning the best and the average solutions among 30 different performance that show the sum number of constraints violated by algorithms. In these sections A_i ($i = 1, 2, \dots, 24$) are algorithms which have been used in mentioned datasets and presented in detail in Appendix 1.

Table 3. Comparison between the proposed algorithm and other algorithms on the BenPaechter dataset

Algorithms	Categories																							
	Small1				Small2				Small3				Small4				Small5				Medium1			
	M	B	M	B	M	B	M	B	M	B	M	B	M	B	M	B	M	B	M	B	M	B	M	B
A_1	0	0	0	0	0	0	0	0	0	0	0	0	245	242	263	161	268	265	184	181	153	151	-	-
A_2	-	6	-	7	-	3	-	3	-	372	-	419	-	359	-	348	-	171	-	1068	-	-	-	-
A_3	-	0	-	0	-	0	-	0	-	0	-	221	-	147	-	246	-	165	-	135	-	529	-	-
A_4	-	3	-	4	-	6	-	6	-	0	-	140	-	130	-	189	-	112	-	141	-	876	-	-
A_5	0	0	0	0	0	0	0	0	0	0	229	227	185	180	238	235	155	142	203	200	-	-	-	-
A_6	-	0	-	0	-	0	-	0	-	0	-	317	-	313	-	357	-	247	-	292	-	-	-	-

A ₇	-	1	-	2	-	0	-	1	-	0	-	146	-	173	-	267	-	169	-	303	-	-	-	-
A ₈	8	-	11	-	8	-	7	-	5	-	199	-	202	-	-	-	177	-	-	-	-	-	-	-
A ₉	-	0	-	3	-	0	-	0	-	0	-	280	-	188	-	249	-	247	-	232	-	-	-	-
A ₁₀	1	-	3	-	1	-	1	-	0	-	195	-	184	-	248	-	164	-	219	-	851	-	-	-
A ₁₁	-	10	-	9	-	7	-	17	-	7	-	243	-	325	-	249	-	285	-	112	-	1138	-	-
A ₁₂	0	0	0	0	0	0	0	0	0	0	143	139	69	62	124	122	101	98	119	116	622	615	-	-
A ₁₃	-	2	-	4	-	2	-	0	-	4	-	254	-	258	-	251	-	321	-	276	-	1027	-	-
A ₁₄	-	0	-	3	-	0	-	0	-	0	-	280	-	188	-	249	-	247	-	232	-	-	-	-
A ₁₅	-	0	-	0	-	0	-	0	-	0	-	180	-	176	-	219	-	150	-	196	-	-	-	-
A ₁₆	-	0	-	0	-	0	-	0	-	0	-	242	-	161	-	265	-	181	-	151	-	-	-	-
A ₁₇	-	0	-	0	-	0	-	0	-	0	-	221	-	147	-	246	-	165	-	130	-	-	-	-
A ₁₈	-	6	-	7	-	3	-	3	-	4	-	372	-	419	-	359	-	348	-	171	-	-	-	-
A ₁₉	-	0	-	0	-	0	-	0	-	0	-	317	-	313	-	357	-	247	-	292	-	-	-	-
A ₂₀	-	1	-	2	-	0	-	1	-	0	-	146	-	173	-	267	-	169	-	303	-	-	-	-
A ₂₁	-	10	-	9	-	7	-	17	-	7	-	243	-	325	-	249	-	285	-	132	-	-	-	-
A ₂₂	0	0	0	0	0	0	0	0	0	0	242	240	258	160	245	242	161	158	126	124	822	801	-	-
A ₂₃	0	0	0	0	0	0	0	0	0	0	131	106	131	107	151	132	93	72	125	107	555	505	524	486
A ₂₄	0	0	0	0	0	0	0	0	0	0	93	84	111	99	154	142	90	84	132	112	603	516	833	557

In the Table 3 at first the performance of algorithms A₁, A₅, A₁₂, A₂₂, A₂₃ and A₂₄ are evaluated on small (1–5) dataset, where the ranking of algorithms would have better performance in terms of efficiency over this dataset so that here A₂₃ and A₂₄ algorithms have good performance. We could obtain the performance of the above algorithms on Medium (1–5) size dataset where the ranking process of algorithms would have better performance in terms of efficiency over this data A₁₂ so that here the efficiency of the algorithm A₂₄ is the best and A₁₅ and A₂₃ algorithms have good performance. Finally, we could obtain the performance of mentioned algorithms on large (1–2) and very large dataset where the ranking process of algorithms would have better performance in terms of efficiency over these two datasets so that in the large dataset, the efficiency of the algorithm A₂₄ is the best and A₁₂ and A₂₃ algorithms have good performance. However, the worst performance belongs to both A₁₁ and A₁₃ algorithms. In general, we find that the performance of the algorithm A₂₃ and A₂₄ (proposed algorithm) in all Categories is better than other algorithms. The studies performed on the issue of run-time of the compared hybrid algorithms reveal that the algorithms A₁₅, A₁₇ and A₂₄ have less average run-time compared to other methods. The run-time of A₁₅ and A₂₄ algorithms is 825s and the 780s for the BenPaechter dataset (average for all instances). This comparison is performed only among some algorithms, since many algorithms do not investigate the run-time factor in their studies.

VI. CONCLUSION AND FUTURE SCOPE

Scheduling is one of the problems so many researches have been worked on it over the years. In this study, a new method was presented for solving timetabling problems of University

courses based on genetic algorithm. In the proposed method, hard and soft constraints are used to determine suitable solutions and the criterion of the distance to feasibility of solutions have also been used. Regarding the large instance, our PGALS was not always able to produce a feasible timetable. Our method like some approaches, cannot always produce a feasible timetable for the large instance. Future work includes further analysis of the contribution of individual components (local search and guided search) toward the performance of PGALS. We believe that the performance of the genetic algorithm for the UCTP can be improved by applying advanced genetic operators and heuristics. Future work includes further analysis of the contribution of individual components (simulated annealing and guided search) toward the performance of PGALS.

APPENDIX 1.

Full names of applied algorithms (A_i) in Table III. A₁: (RIICN: Randomize Iterative Improvement with Composite nears Algorithm [7]), A₂: (GBHH: Graph Based Hyper Heuristic [26]), A₃: (HEA: Hybrid Evolutionary Algorithm [27]), A₄: (NLGD: Non-Linear Great Deluge Algorithm [28]), A₅: (MA: Memetic Algorithm [29]), A₆: (VNS: Variable Neighborhood Search [7]), A₇: (THHS: Taboo Based Hyper Heuristics [30]), A₈: (LS: Local Search [31]), A₉: (EA: Evolutionary Algorithm [32]), A₁₀: (AA: Ant Colony Optimization Algorithm [33]), A₁₁: (FA: Fuzzy Approach [34]), A₁₂: (EGSGA: Extended Guided Search with Genetic Algorithm [35]), A₁₃: (GA w LS: Genetic Algorithm with Local Search [36]), A₁₄: (GA: Genetic Algorithm [32]), A₁₅: (HGA : Hybrid Genetic Algorithm [37]), A₁₆: (RIIA: Randomised Iterative Improvement Algorithm [30]), A₁₇: (HEA: Hybrid Evolutionary Approach [9]), A₁₈: (GHH: Graph-based Hyper Heuristic [26]), A₁₉: (VNS : Variable

Neighborhood Search [38]), A₂₀: (TSH : Tabu-Search Hyperheuristic [39]), A₂₁: (FMH: Fuzzy Multiple Heuristic [30], A₂₂: (GSGA: Guided Search Genetic Algorithm [35]), A₂₃: (HA: Hybrid Algorithm [39]), A₂₄: (PGALS: Parallel Improved Genetic Algorithm and Local Search (The proposed algorithm)).

REFERENCES

- [1] J. A. Soria-Alcaraz, G. Ochoa, J. Swan, M. Carpio, H. Puga, and E. K. Burke, "Effective learning hyper-heuristics for the course timetabling problem," *European Journal of Operational Research*, Vol. 238, Issue. 1, pp. 77-86, 2014.
- [2] E. K. Burke, J. H. Drake, B. McCollum, and E. Özcan, "Comments on: An overview of curriculum-based course timetabling," *Top*, Vol. 23, Issue. 2, pp. 355-358, 2015.
- [3] M. Ghalegholabi and A. Rezaeipannah, "Intrusion Detection System Using Genetic Algorithm and Data Mining Techniques Based on the Reduction Features", *International Journal of Computer Applications Technology and Research*, Vol. 6, Issue. 11, pp. 461-466, 2016.
- [4] J. Makkar, H. Monga, and S. Baglha, "Reduction of Inter-Symbol Interference using Artificial Neural Network System in Multicarrier OFDM System," *International Journal of Scientific Research in Computer Sciences and Engineering*, Vol. 2, Issue. 5, pp. 553-558, 2017.
- [5] R. Karimpour, M. Khayyambashi and N. Movahhedinia, "Load Balancing in Grid Computing Using Ant Colony Algorithm and Max-min Technique", *Malaysian Journal of Computer Science*, Vol. 29, Issue. 3, pp. 196-206, 2016.
- [6] A. Alkan, and E. Ozcan, "Memetic algorithms for timetabling," In *Evolutionary Computation*, IEEE, Vol. 3, pp. 1796-1802, 2003.
- [7] C. H. Aladag, G. Hocaoglu and M. Basaran, "The effect of neighborhood structures on tabu search algorithm in solving course timetabling problem", *Expert Systems with Applications*, Vol. 36, Issue. 10, pp. 12349-12356, 2009.
- [8] M. Nandhini, and Kanmani, "A survey of simulated annealing methodology for university course timetabling," *International Journal of Recent Trends in Engineering*, Vol. 1, Issue. 2, 255-262, 2009.
- [9] S. Abdullah, H. Turabieh, B. McCollum and P. McMullan, "A hybrid metaheuristic approach to the university course timetabling problem", *Journal of Heuristics*, Vol. 18, Issue. 1, pp. 1-23, 2012.
- [10] R. Chen and H. Shih, "Solving University Course Timetabling Problems Using Constriction Particle Swarm Optimization with Local Search", *Algorithms*, Vol. 6, Issue. 2, pp. 227-244, 2013.
- [11] M. Al-Betar, A. Khader and M. Zaman, "University Course Timetabling Using a Hybrid Harmony Search Metaheuristic Algorithm", *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, Vol. 42, Issue. 5, pp. 664-681, 2012.
- [12] R. Bai, E. Burke, G. Kendall, J. Li and B. McCollum, "A Hybrid Evolutionary Approach to the Nurse Rostering Problem", *IEEE Transactions on Evolutionary Computation*, Vol. 14, Issue. 4, pp. 580-590, 2010.
- [13] S. Ait Haddadene, N. Labadie and C. Prodhon, "A GRASP $\times$ ILS for the vehicle routing problem with time windows, synchronization and precedence constraints", *Expert Systems with Applications*, Vol. 66, pp. 274-294, 2016.
- [14] N. Pillay, "A survey of school timetabling research", *Annals of Operations Research*, Vol. 218, Issue. 1, pp. 261-293, 2014.
- [15] A. V. Moura and R. Scaraficci, "A GRASP strategy for a more constrained School Timetabling Problem", *International Journal of Operational Research*, Vol. 7, Issue. 2, pp. 152-160, 2010.
- [16] Bolaji, A. Khader, M. Al-Betar and M. Awadallah, "University course timetabling using hybridized artificial bee colony with hill climbing optimizer", *Journal of Computational Science*, Vol. 5, Issue. 5, pp. 809-818, 2014.
- [17] S. Daskalaki, T. Birbas and E. Housos, "An integer programming formulation for a case study in university timetabling", *European Journal of Operational Research*, Vol. 153, Issue. 1, pp. 117-135, 2004.
- [18] G. da Fonseca, H. Santos, T. Toffolo, S. Brito and M. Souza, "GOAL solver: a hybrid local search based solver for high school timetabling", *Annals of Operations Research*, Vol. 239, Issue. 1, pp. 77-97, 2014.
- [19] L. Ahmed, E. Özcan and A. Kheiri, "Solving high school timetabling problems worldwide using selection hyper-heuristics", *Expert Systems with Applications*, Vol. 42, Issue. 13, pp. 5463-5471, 2015.
- [20] S. Al-Yakoob and H. Sherali, "Mathematical models and algorithms for a high school timetabling problem", *Computers & Operations Research*, Vol. 61, pp. 56-68, 2015.
- [21] R. Lewis, and J. Thompson, "Analysing the effects of solution space connectivity with an effective metaheuristic for the course timetabling problem," *European Journal of Operational Research*, Vol. 240, Issue. 3, pp. 637-648, 2015.
- [22] E. K. Burke, M. Gendreau, M. Hyde, G. Kendall, G. Ochoa, E. Özcan, and R. Qu, "Hyper-heuristics: A survey of the state of the art," *Journal of the Operational Research Society*, Vol. 64, Issue. 12, pp. 1695-1724, 2013.
- [23] Datta, K. Deb, C. M. Fonseca, "Multi-objective evolutionary algorithm for university class timetabling problem," In *Evolutionary scheduling*, Springer, Berlin, Heidelberg, pp. 197-236, 2017.
- [24] S. Abdullah, H. Turabieh, B. McCollum, and P. McMullan, "A multi-objective post enrolment course timetabling problems: a new case study," In *IEEE congress on evolutionary computation*, IEEE, pp. 1-7, 2010.
- [25] T. Thepphakorn, P. Pongcharoen, and S. Vitayasak, "A New Multiple Objective Cuckoo Search for University Course Timetabling Problem," In *International Workshop on Multi-disciplinary Trends in Artificial Intelligence*, Springer, Cham, pp. 196-207, 2016.
- [26] E. K. Burke, B. McCollum, A. Meisels, S. Petrovic, and R. Qu, "A graph-based hyper-heuristic for educational timetabling problems," *European Journal of Operational Research*, Vol. 176, Issue. 1, pp. 177-192, 2007.
- [27] M. Rogalska, W. Bożejko, and Z. Hejducki, "Time/cost optimization using hybrid evolutionary algorithm in construction project scheduling," *Automation in Construction*, Vol. 18, Issue. 1, pp. 24-31, 2008.
- [28] S. Kifah, and S. Abdullah, "An adaptive non-linear great deluge algorithm for the patient-admission problem," *Information Sciences*, Vol. 295, pp. 573-585, 2015.
- [29] Y. Lei, M. Gong, L. Jiao, and Y. Zuo, "A memetic algorithm based on hyper-heuristics for examination timetabling problems," *International Journal of Intelligent Computing and Cybernetics*, Vol. 8, Issue. 2, pp. 139-151, 2015.
- [30] H. Babaei, J. Karimpour, and A. Hadidi, "A survey of approaches for university course timetabling problem," *Computers & Industrial Engineering*, Vol. 86, 43-59, 2015.
- [31] J. A. Soria-Alcaraz, E. Özcan, J. Swan, G. Kendall, and M. Carpio, "Iterated local search using an add and delete hyper-heuristic for university course timetabling," *Applied Soft Computing*, Vol. 40, 581-593, 2016.

- [32] G. N. Beligiannis, C. N. Moschopoulos, G. P. Kaperonis, and S. D. Likothanassis, "Applying evolutionary computation to the school timetabling problem: The Greek case," *Computers & Operations Research*, Vol. **35**, Issue. **4**, pp. **1265-1280**, **2008**.
- [33] Nothegger, A. Mayer, A. Chwatal, and G. R. Raidl, "Solving the post enrolment course timetabling problem by ant colony optimization," *Annals of Operations Research*, Vol. **194**, Issue. **1**, pp. **325-339**, **2012**.
- [34] F. Cavdur, and M. Kose, "A fuzzy logic and binary-goal programming-based approach for solving the exam timetabling problem to create a balanced-exam schedule," *International Journal of Fuzzy Systems*, Vol. **18**, Issue. **1**, pp. **119-129**, **2016**.
- [35] S. Yang, and S. N. Jat, "Genetic algorithms with guided and local search strategies for university course timetabling," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, Vol. **41**, Issue. **1**, pp. **93-106**, **2011**.
- [36] H. Ishibuchi, T. Yoshida, and T. Murata, "Balance between genetic search and local search in memetic algorithms for multiobjective permutation flowshop scheduling," *IEEE transactions on evolutionary computation*, Vol. **7**, Issue. **2**, pp. **204-223**, **2003**.
- [37] X. Feng, Y. Lee, and I. Moon, "An integer program and a hybrid genetic algorithm for the university timetabling problem," *Optimization Methods and Software*, Vol. **32**, Issue. **3**, pp. **625-649**, **2017**.
- [38] N. Mladenović, M. Dražić, V. Kovačević-Vujčić, and M. Čangalović, "General variable neighborhood search for the continuous optimization," *European Journal of Operational Research*, Vol. **191**, Issue. **3**, pp. **753-770**, **2008**.
- [39] E. K. Burke, G. Kendall, and E. Soubeiga, "A Tabu-Search Hyperheuristic for Timetabling and Rostering," *Springer Journal of Heuristics*, pp. **451-470**, **2003**.

Authors Profile

Mr. Amin Rezaeipناه received her B.E. in Computer Science & Engineering from University Ferdows College of Engineering, Mashhad, Iran in 2010 and M.E. in Software Engineering and Artificial intelligence from Shiraz University, Shiraz, Iran in 2013. He is a postdoctoral researcher at Ph.D. He is also a graduate teaching assistant at Rahjuyan Danesh Borazjan University. His main research interests are recommender systems, social network analysis, WSN and large-scale data mining.



Ms. Zahra Abshirini received her B.E. in chemical engineering from Shahid Rajaee Teacher Training University, Tehran, Iran in 2009 and M.E. in chemical engineering from Payame Noor University, Bushehr, Iran in 2013. He is a postdoctoral researcher at Ph.D. He is also a graduate teaching assistant at Education. His main research interests are synthesis pyrazole derivatives, production of chromones and nano catalysts.



Mr. Milad Boshkani Zadeh received received her B.E. in Computer Science & Engineering from Islamic Azad University - Bushehr Branch, Bushehr, Iran in 2008 and M.E. in Information Technology from Islamic Azad University - Ahram Branch, Bushehr, Iran in 2011. His main research interests are data mining, Parallel computing and Simulation Optimization.

